Rapport des Exercices CUDA

Jacques Gastebois

17 avril 2026

Table des matières

1 Configuration du GPU **2**

2 Exercice 1 : Informations sur le GPU **2**
2.1 Code Source 2
2.2 Observations 3

3 Exercice 2 : Hello World et Grilles 2D/3D **3**
3.1 Code Source 3
3.2 Observations 4

4 Exercice 3 : Calcul dÍndices Globaux Uniques en 3D **4**
4.1 Code Source 4
4.2 Observations 6

5 Exercice 4 : Divergence de Threads vs Warps **6**
5.1 Code Source (Benchmark) 6
5.2 Résultats et Analyse 8

6 Exercice 5 : Scan Parallèle **9**
6.1 Code Source (Kernels principaux) 9
6.2 Résultats et Analyse 10

7 Exercice 6 : Optimisation de lÉnsemble de Julia **10**
7.1 Code Source (Kernels) 11
7.2 Résultats et Analyse 11
7.3 Conclusion 12

1 Configuration du GPU

La configuration du GPU distant utilisé pour ces tests est la suivante :
- NVIDIA Tesla M60
- 16 multiprocesseurs
- Compute capability 5.2
- 8 GB de mémoire globale

2 Exercice 1 : Informations sur le GPU

Lóbjectif de cet exercice est de récupérer et dáfficher les caractéristiques du GPU disponible.

2.1 Code Source

```
#include "cuda_runtime.h" // pour les fonctions CUDA
#include "device_launch_parameters.h" // pour les param tres de lancement des kernels
#include <stdio.h> // pour les fonctions d\'entr e/sortie

void query_device()
{
    int deviceCount = 0; // initialisation du nombre de dispositifs CUDA
    cudaGetDeviceCount(&deviceCount); // r cup re le nombre de dispositifs CUDA disponibles
    if (deviceCount == 0) // si aucun dispositif CUDA n\'est disponible
    {
        printf("No CUDA support device found"); // affiche un message d\'erreur
    }
    int devNo = 0; // initialisation du num ro du dispositif
    cudaDeviceProp iProp; // d claration de la structure cudaDeviceProp
    cudaGetDeviceProperties(&iProp, devNo); // r cup re les propri t s du dispositif
    printf("Device %d: %s\n", devNo, iProp.name);
    printf(" Number of multiprocessors: %d\n", iProp.multiProcessorCount); // affiche le nombre de multiprocesseurs
    printf(" clock rate : %d\n", iProp.clockRate); // affiche la fr quence d\'horloge
    printf(" Compute capability : %d.%d\n", iProp.major, iProp.minor); // affiche la version de la carte graphique
    printf(" Total amount of global memory: %4.2f KB\n", iProp.totalGlobalMem / 1024.0); // affiche la quantit  de m moire globale
    printf(" Total amount of constant memory: %4.2f KB\n", iProp.totalConstMem / 1024.0); // affiche la quantit  de m moire constante
    printf(" Total amount of shared memory per block: %4.2f KB\n", iProp.sharedMemPerBlock / 1024.0); // affiche la quantit  de m moire partag e par bloc
    printf(" Total amount of shared memory per MP: %4.2f KB\n", iProp.sharedMemPerMultiprocessor / 1024.0); // affiche la quantit  de m moire partag e par multiprocesseur
    printf(" Total number of registers available per block: %d\n", iProp.regsPerBlock); // affiche le nombre de registres disponibles par bloc
    printf(" Warp size: %d\n", iProp.warpSize); // affiche la taille du warp
    printf(" Maximum number of threads per block: %d\n", iProp.maxThreadsPerBlock); // affiche le nombre maximum de threads par bloc
    printf(" Maximum number of threads per multiprocessor: %d\n", iProp.maxThreadsPerMultiProcessor); // affiche le nombre maximum de threads par multiprocesseur
    printf(" Maximum number of warps per multiprocessor: %d\n", iProp.maxThreadsPerMultiProcessor / 32); // affiche le nombre maximum de warps par multiprocesseur
```

```
    printf(" Maximum Grid size : (%d,%d,%d)\n", iProp.maxGridSize[0], iProp.maxGridSize[1], iProp.maxGridSize[2]); // affiche la taille maximale de la grille
    printf(" Maximum block dimension : (%d,%d,%d)\n", iProp.maxThreadsDim[0], iProp.maxThreadsDim[1], iProp.maxThreadsDim[2]); // affiche la taille maximale d\'un bloc
}

int main()
{
    query_device(); // appelle la fonction query_device
    return 0; // retourne 0 pour indiquer que le programme s\'est termin  avec succ s
}
```

Listing 1 – Code de léxercice 1 (exercice1.cu)

2.2 Observations

Cet exercice permet de vérifier la présence dún GPU compatible CUDA et dóbtenir des informations clés telles que :

- Fréquence dhorloge : 1.17 GHz
- Nombre de threads maximum par bloc : 1024
- Taille dún warp : 32 threads
- Mémoire partagée par bloc : 48 KB

Ces informations sont cruciales pour dimensionner correctement les kernels CUDA.

3 Exercice 2 : Hello World et Grilles 2D/3D

Cet exercice introduit le lancement de kernels simples et lútilisation des identifiants de threads et de blocs dans des grilles de différentes dimensions.

3.1 Code Source

```
#include "cuda_runtime.h"
#include "device_launch_parameters.h"
#include <stdio.h>

__global__ void hello_kernel() // d claration du kernel qui sera execut  sur la carte graphique
{
    printf("Hello cuda world \n");
}

__global__ void hello_kernel_3d() // nouveau kernel pour tester les configurations 2D/3D
{
    // indices 3D du thread dans le bloc
    int tx = threadIdx.x;
    int ty = threadIdx.y;
    int tz = threadIdx.z;

    // indices 3D du bloc dans la grille
    int bx = blockIdx.x;
    int by = blockIdx.y;
    int bz = blockIdx.z;

    printf("Hello cuda world \n");
```

```
    printf("        thread(%d,%d,%d) du bloc(%d,%d,%d)\n", tx, ty, tz, bx, by, bz
        );
}

int main() // fonction principale du programme execut  par le CPU
{
    // test 1: hello world simple
    printf("hello from main \n"); // affiche un message de bienvenue
    hello_kernel<<<1, 1>>>(); // lance le kernel hello_kernel
    cudaDeviceSynchronize(); // attend que le kernel soit termin

    printf("\  n     test des configurations 2D et 3D\n\n");

    // configuration 2D (2x2 threads par bloc, grille 2x2)
    printf("        config 2D: 2x2 threads par bloc, grille 2x2\n");
    dim3 block2D(2, 2);
    dim3 grid2D(2, 2);
    hello_kernel_3d<<<grid2D, block2D>>>();
    cudaDeviceSynchronize();

    printf("\n");

    // configuration 3D (2x2x2 threads par bloc, grille 2x2x2)
    printf("        config 3D: 2x2x2 threads par bloc, grille 2x2x2\n");
    dim3 block3D(2, 2, 2);
    dim3 grid3D(2, 2, 2);
    hello_kernel_3d<<<grid3D, block3D>>>();
    cudaDeviceSynchronize();

    cudaDeviceReset(); // r initialise le dispositif CUDA
    return 0;
}
```

Listing 2 – Code de léxercice 2 (exercice2.cu)

3.2 Observations

Léxercice montre comment les threads sont identifiés dans des configurations 2D et 3D. Les points clés sont :

- Lórdre déxécution des threads et des blocs nést pas déterministe, ce qui est fondamental en programmation parallèle.
- Les indices ‘z‘ des threads et blocs sont à 0 lors de lútilisation de ‘dim3‘ avec seulement deux arguments pour une grille 2D.

4 Exercice 3 : Calcul dÍndices Globaux Uniques en 3D

Cet exercice se concentre sur le calcul dún identifiant global unique pour chaque thread dans une grille 3D, une technique essentielle pour adresser des données dans de grands tableaux.

4.1 Code Source

```
#include "cuda_runtime.h"
#include "device_launch_parameters.h"
#include <stdio.h>

__global__ void unique_gid_calculation(int * data)
{
    int tid = threadIdx.x; // indice du thread dans le bloc
```

```
    int gid = blockIdx.x * blockDim.x + threadIdx.x; // calcul de l\'indice global
    printf("blockIdx.x : %d, blockIdx.y : %d, tid : %d, gid : %d - data : %d \n",
        blockIdx.x, blockIdx.y, tid, gid, data[gid]);// affichage des indices et de la valeur
}

__global__ void unique_gid_calculation_3d(int * data)
{
    // indices 3D du thread dans le bloc
    int tx = threadIdx.x;
    int ty = threadIdx.y;
    int tz = threadIdx.z;

    // indices 3D du bloc dans la grille
    int bx = blockIdx.x;
    int by = blockIdx.y;
    int bz = blockIdx.z;

    // calcul de l\'indice global
    int gid = tx + (ty * blockDim.x) + (tz * blockDim.x * blockDim.y) +
              (bx * blockDim.x * blockDim.y * blockDim.z) +
              (by * gridDim.x * blockDim.x * blockDim.y * blockDim.z) +
              (bz * gridDim.x * gridDim.y * blockDim.x * blockDim.y * blockDim.z);

    if (gid < 64) { // protection contre les depassements
        printf("       bloc(%d,%d,%d) thread(%d,%d,%d) gid:%d - data:%d\n",
            bx, by, bz, tx, ty, tz, gid, data[gid]);
    }
}

int main()
{
    int array_size = 64;
    int array_byte_size = sizeof(int) * array_size;
    int* h_data = (int*)malloc(array_byte_size); // allocation dynamique sur le cpu

    // initialisation des donnees
    for(int i = 0; i < array_size; i++) {
        h_data[i] = i;
    }

    int * d_data;
    cudaMalloc((void**)&d_data, array_byte_size); // allocation de la memoire sur le gpu
    cudaMemcpy(d_data, h_data, array_byte_size, cudaMemcpyHostToDevice);// copie des donnees de l\'hote vers le gpu

    // configuration 3d: bloc 2x2x2, grille 2x2x2
    dim3 block(2,2,2); // 8 threads par bloc
    dim3 grid(2,2,2);  // 8 blocs au total
    unique_gid_calculation_3d<<< grid, block >>>(d_data);
    cudaDeviceSynchronize();

    // liberation de la memoire
    free(h_data);
    cudaFree(d_data);
    cudaDeviceReset();
    return 0;
}
```

Listing 3 – Code de léxercice 3 (exercice3.cu)

4.2 Observations

Les résultats confirment que la formule de calcul du GID (Global ID) génère bien des indices uniques pour chaque thread dans la grille 3D. Cela permet un adressage correct des données. On observe également la gestion de la mémoire avec les allocations sur CPU (‘malloc‘), GPU (‘cudaMalloc‘), les copies (‘cudaMemcpy‘) et les libérations (‘free‘, ‘cudaFree‘).

5 Exercice 4 : Divergence de Threads vs Warps

Lóbjectif est d´tudier límpact de la divergence des threads au sein dún warp et entre warps sur les performances.

5.1 Code Source (Benchmark)

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include "cuda.h"
#include "cuda_runtime.h"
#include "device_launch_parameters.h"
#include <math.h>

// version avec divergence au niveau des threads
__global__ void thread_divergence()
{
    int gid = blockIdx.x * blockDim.x + threadIdx.x;
    float a = 1.0f;
    float b = 2.0f;
    float result = 1.0f;

    // divergence tous les 2 threads
    if (gid % 2 == 0)
    {
        // branche lourde avec beaucoup de calculs
        for(int i = 0; i < 100; i++) {
            result = result * sinf(a) + cosf(b);
            a += result * 0.1f;
            b -= result * 0.05f;
        }
    }
    else
    {
        // branche legere
        result = a + b;
    }
}

// version avec divergence au niveau des warps
__global__ void warp_divergence()
{
    int gid = blockIdx.x * blockDim.x + threadIdx.x;
    float a = 1.0f;
    float b = 2.0f;
    float result = 1.0f;

    // divergence tous les 2 warps
```

```
    int warp_id = gid / 32;
    if (warp_id % 2 == 0)
    {
        // branche lourde avec beaucoup de calculs
        for(int i = 0; i < 100; i++) {
            result = result * sinf(a) + cosf(b);
            a += result * 0.1f;
            b -= result * 0.05f;
        }
    }
    else
    {
        // branche legere
        result = a + b;
    }
}

// fonction pour mesurer le temps d\'execution
float benchmark_kernel(void (*kernel)(), dim3 grid, dim3 block, int iterations)
{
    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);

    // warm up
    kernel<<<grid, block>>>();
    cudaDeviceSynchronize();

    // mesure du temps
    cudaEventRecord(start);

    for(int i = 0; i < iterations; i++) {
        kernel<<<grid, block>>>();
    }

    cudaEventRecord(stop);
    cudaEventSynchronize(stop);

    float ms;
    cudaEventElapsedTime(&ms, start, stop);

    cudaEventDestroy(start);
    cudaEventDestroy(stop);

    return ms / iterations; // temps moyen par iteration
}

int main()
{
    // ouvre le fichier CSV pour les resultats
    FILE* csv = fopen("benchmark_results.csv", "w");
    fprintf(csv, "size,threads,warps\n"); // en-tete csv

    // tableaux pour stocker les resultats
    const int num_tests = 15; // de 2^10 a 2^24
    int sizes[num_tests];
    float times_thread[num_tests];
    float times_warp[num_tests];
    int num_blocks[num_tests];

    // teste differentes tailles de tableau (de 2^10 a 2^24)
    for(int i = 0; i < num_tests; i++) {
        int power = i + 10;
```

```
        int size = 1 << power;
        sizes[i] = size;

        dim3 block_size(128);
        dim3 grid_size((size + block_size.x - 1) / block_size.x);
        num_blocks[i] = grid_size.x;

        int iterations = 1000;
        if (power > 20) iterations = 100; // reduit le nombre d\'iterations pour les grands tableaux

        times_thread[i] = benchmark_kernel(thread_divergence, grid_size, block_size, iterations);
        times_warp[i] = benchmark_kernel(warp_divergence, grid_size, block_size, iterations);

        // sauvegarde dans le csv
        fprintf(csv, "%d,%.6f,%.6f\n", size, times_thread[i], times_warp[i]);
    }

    fclose(csv);

    // affiche les resultats a la fin
    printf("\n resultats du benchmark:\n");
    printf("\n");
    printf("    taille          blocs        threads ms     warps ms       ratio\n");
    printf("\n");

    for(int i = 0; i < num_tests; i++) {
        float ratio = times_thread[i] / times_warp[i];
        printf("    %9d       %8d     %8.3f      %8.3f      %7.3f    \n",
               sizes[i], num_blocks[i], times_thread[i], times_warp[i], ratio);
    }

    printf("\n");
    printf("\n resultats sauvegardes dans benchmark_results.csv\n");

    cudaDeviceReset();
    return 0;
}
```

Listing 4 – Code de léxercice 4 (exercice4$_b$*enchmark.cu*)

5.2 Résultats et Analyse

Le benchmark compare deux scénarios de divergence : lún où les threads adjacents prennent des chemins différents, et láutre où des warps entiers prennent des chemins différents. Les résultats montrent que :

- Pour des opérations simples, la différence de performance est minime. Les GPU modernes gèrent efficacement la divergence simple.
- Un gain modeste (jusqu´ 8% pour la taille de 524K éléments) est observé pour la version avec divergence au niveau des warps lorsque les branches de code ont des charges de travail très différentes.

— Límpact de la divergence devient plus significatif avec des opérations plus complexes et des accès mémoire divergents.

6 Exercice 5 : Scan Parallèle

Cet exercice implémente et compare différentes versions dún algorithme de scan parallèle (somme préfixe).

6.1 Code Source (Kernels principaux)

```
//        version 2: work-efficient scan (algorithme de hillis & steele)
__global__ void scan_workefficient(int *input, int *output, int size)
{
    extern __shared__ int temp[];
    int tid = threadIdx.x;
    int gid = blockIdx.x * blockDim.x + threadIdx.x;

    // charge les donnees dans la memoire partagee
    if (gid < size)
        temp[tid] = input[gid];
    else
        temp[tid] = 0;
    __syncthreads();

    // scan avec doublement de la distance
    for (int offset = 1; offset < blockDim.x; offset *= 2)
    {
        int t = (tid >= offset) ? temp[tid - offset] : 0;
        __syncthreads();
        if (tid >= offset)
            temp[tid] += t;
        __syncthreads();
    }

    // ecrit le resultat
    if (gid < size)
        output[gid] = temp[tid];
}

//        version 3: scan par blocs avec propagation (algorithme de blelloch)
__global__ void scan_block(int *input, int *output, int *block_sums, int size)
{
    extern __shared__ int temp[];
    int tid = threadIdx.x;
    int gid = blockIdx.x * blockDim.x + threadIdx.x;

    // phase 1: scan local dans chaque bloc
    if (gid < size)
        temp[tid] = input[gid];
    else
        temp[tid] = 0;
    __syncthreads();

    // reduction montante (up-sweep)
    for (int stride = 1; stride < blockDim.x; stride *= 2)
    {
        int index = (tid + 1) * 2 * stride - 1;
        if (index < blockDim.x)
            temp[index] += temp[index - stride];
        __syncthreads();
    }
```

```
    // stocke la somme du bloc
    if (tid == 0 && block_sums != NULL)
        block_sums[blockIdx.x] = temp[blockDim.x - 1];

    // reduction descendante (down-sweep)
    for (int stride = blockDim.x / 4; stride > 0; stride /= 2)
    {
        int index = (tid + 1) * 2 * stride - 1;
        if (index + stride < blockDim.x)
        {
            temp[index + stride] += temp[index];
        }
        __syncthreads();
    }

    // ecrit le resultat
    if (gid < size)
        output[gid] = temp[tid];
}

//        kernel pour propager les sommes des blocs
__global__ void propagate_block_sums(int *output, int *block_sums, int size, int
     block_size)
{
    int gid = blockIdx.x * blockDim.x + threadIdx.x;
    if (gid < size && blockIdx.x > 0)
    {
        output[gid] += block_sums[blockIdx.x - 1];
    }
}
```

Listing 5 – Kernels de scan parallèle (exercice5.cu - extraits)

6.2 Résultats et Analyse

Le benchmark compare une version CPU, une version "work-efficient" (Hillis Steele) et une version par blocs (Blelloch).

- Pour les petites tailles de données (< 4K éléments), le CPU est compétitif en raison de lóverhead du GPU.
- Pour les tailles moyennes et grandes, les versions GPU sont significativement plus rapides. La version "work-efficient" est généralement la plus performante, offrant une accélération allant jusqu´ 21x par rapport au CPU pour 16M éléments.
- Le choix de lálgorithme de scan dépend fortement de la taille des données.

Le tableau suivant résume les performances pour quelques tailles de données (temps en ms) :

Taille	CPU (ms)	Work-efficient (ms)	Block-based (ms)
1024	0.006	0.172	0.806
32768 (32K)	0.208	0.025	0.047
16777216 (16M)	100.665	4.743	8.886

Table 1 – Performances des algorithmes de Scan (temps en ms)

7 Exercice 6 : Optimisation de lÉnsemble de Julia

Cet exercice explore différentes optimisations pour le calcul de lénsemble de Julia en CUDA.

7.1 Code Source (Kernels)

```
//        kernel cuda basique
__global__ void generate_julia_kernel_v1(Color* pixels, double cRe, double cIm)
{
    const int idx = blockIdx.x * blockDim.x + threadIdx.x;
    const int x = idx % WIDTH;
    const int y = idx / WIDTH;

    if(y < HEIGHT) { /* ... calcul et coloration ... */ }
}

//        kernel cuda optimise avec pre-calcul
__global__ void generate_julia_kernel_v2(Color* pixels, double cRe, double cIm)
{
    const int idx = blockIdx.x * blockDim.x + threadIdx.x;
    const int x = idx % WIDTH;
    const int y = idx / WIDTH;

    if(y < HEIGHT) {
        // pre-calcul des carres
        double zRe2 = zRe * zRe;
        double zIm2 = zIm * zIm;
        /* ... calcul et coloration ... */
    }
}

//        kernel cuda optimise avec memoire partagee
__global__ void generate_julia_kernel_v3(Color* pixels, double cRe, double cIm)
{
    __shared__ Color tile[TILE_SIZE * TILE_SIZE];

    const int tx = threadIdx.x % TILE_SIZE;
    const int ty = threadIdx.x / TILE_SIZE;

    const int x = blockIdx.x * TILE_SIZE + tx;
    const int y = blockIdx.y * TILE_SIZE + ty;

    if(x < WIDTH && y < HEIGHT) {
         /* ... calcul et coloration ... */
        tile[ty * TILE_SIZE + tx] = color;
        __syncthreads();
        pixels[y * WIDTH + x] = tile[ty * TILE_SIZE + tx];
    }
}
```

Listing 6 – Kernels pour lénsemble de Julia (exercice6.cu - extraits)

Note : Le code complet du calcul de Julia et de la coloration a été omis pour la concision.

7.2 Résultats et Analyse

Plusieurs versions du calcul de lénsemble de Julia ont été testées sur une image de 800x600 pixels.

Version	Temps (ms)	Accélération
CPU	332.00	1.00x
GPU v1 (basique)	35.02	9.49x
GPU v2 (pré-calcul)	34.49	9.64x
GPU v3 (mémoire partagée)	32.15	10.34x

Table 2 – Performances du calcul de lénsemble de Julia (800x600)

Améliorations relatives entre versions GPU :
— v1 → v2 (pré-calcul des carrés) : +1.5%
— v2 → v3 (utilisation mémoire partagée) : +7.2%
— Total v1 → v3 : +8.8%

Précision : Toutes les versions GPU ont montré une excellente précision par rapport à la version CPU, avec seulement 0.45% de pixels différents, principalement dus aux arrondis en calcul flottant. Les différences moyennes par canal de couleur étaient très faibles (∼0.35 sur 255).

Emplacement pour julia_gpu_v3.png

Figure 1 – Visualisation de lénsemble de Julia (800x600) généré par la version GPU v3.

7.3 Conclusion

La version utilisant la mémoire partagée (v3) offre les meilleures performances pour le calcul de lénsemble de Julia, avec une accélération de plus de 10x par rapport au CPU. Les optimisations telles que le pré-calcul et lútilisation judicieuse de la mémoire partagée contribuent de manière significative à lámélioration des performances, tout en maintenant une haute précision des résultats.

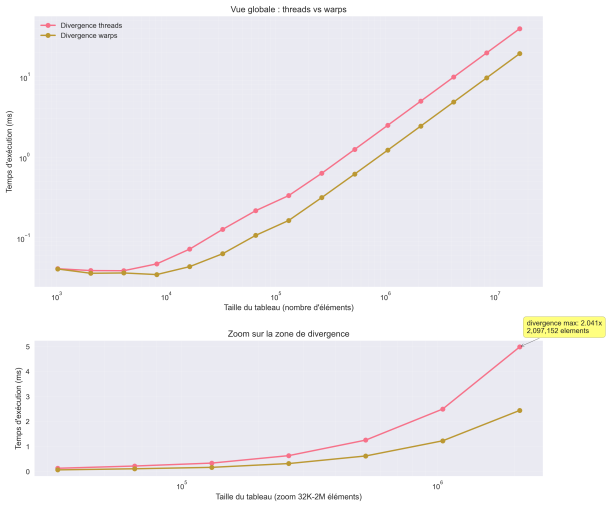

FIGURE 2 –

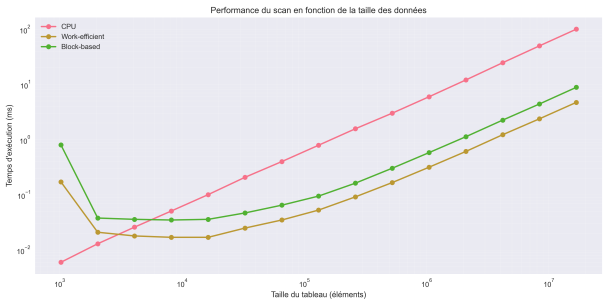

FIGURE 3 –